1. Understanding Diffusion

1.1 Forward diffusion (what happens)

In the forward diffusion process, we start from a clean MNIST image (for example, a handwritten “2”) and progressively add small amounts of Gaussian noise over many time steps. In my notebook’s “noise progression” plot, the first panel is a clean digit; as the percentage of noise increases (20%, 40%, 60%, 80%), the strokes get grainier, parts of the digit fade out, and by the highest noise levels the structure is almost gone and the image resembles random static.

1.2 Why add noise gradually, not all at once?

If we added all the noise in a single step, the mapping from pure noise directly to a clean image would be extremely hard to learn. By diffusing gradually, the reverse problem decomposes into many easier sub-tasks: at each step the model only needs to predict the *small* noise component that was just added, conditioned on the timestep. During training I sample a random timestep t, apply the forward equation, and train the U-Net to predict the noise ε. This local, incremental denoising makes the optimization stable and allows the model to learn a high-quality generative process.

1.3 When does the image become recognizable in reverse diffusion?

In my visualize_generation_steps results, when going from noise back to a digit, the samples look like pure noise at the very first steps, then faint blobs, and only after a certain fraction of the denoising steps do they “snap” into a clear digit. Roughly, the digit became recognizable around the middle of the reverse process—visually around 40–60% of the way through denoising. For simple digits like “1” or “7” the shape emerged a bit earlier, while more complex or curly digits like “8” and “9” needed more steps before they were clearly distinguishable.

2. Model Architecture

2.1 Why U-Net is well-suited for diffusion

The U-Net architecture is ideal for diffusion because it preserves spatial detail while still capturing global context. The encoder (down blocks) progressively reduces resolution and increases channels, which allows it to understand high-level structure of the digit. The decoder (up blocks) then upsamples back to the original resolution. Diffusion denoising is a per-pixel prediction problem: the model must decide, for each pixel, how much of the current value is signal vs. noise. U-Net's combination of multi-scale features and full-resolution output is better for this than a simple CNN that only downsamples, which tends to lose precise location information and produce blurry outputs.

2.2 Skip connections and their importance

Skip connections link feature maps from the encoder directly to the decoder at matching resolutions. In my code, before each down sample I saved the feature map into skip connections, and then in each UpBlock I concatenated the up sampled tensor with the appropriate skip tensor along the channel dimension. These connections allow low-level information (edges, stroke thickness, digit position) to bypass the deep bottleneck and be directly available during reconstruction. Without skips, the bottleneck would have to compress all spatial detail into a small latent vector, which often yields washed out digits. With skips, my generated digits had much sharper edges and recognizable shapes, even at 28×28 resolution.

2.3 How class conditioning works in my model

My model is class-conditional: it can generate a requested digit 0–9. During training and

sampling, I take the label c and convert it to a one-hot vector of length 10. This vector passes through EmbedBlock, a small MLP that outputs a channel-wise embedding reshaped to $[B, C, 1, 1]$. In the U-Net's bottleneck I add this class embedding (multiplied by c_mask for classifier-free guidance) to the feature map along with the time embedding. Intuitively, this shifts the internal representation so that the denoiser "knows" which digit it is supposed to form from the noisy starting point. For example, the same noise input can be turned into a "3" or a "7" depending on which class embedding I add.

3. Training Analysis

3.1 What the loss value tells us

The loss I used is MSE between the predicted noise and the true noise added by the forward process. A high loss means the model's noise estimate is far from correct, i.e., it cannot accurately undo the corruption. Over training, both training and validation losses steadily decreased from around ~0.035–0.04 down to roughly 0.023–0.024. This indicates the U-Net became significantly better at matching the exact noise at each timestep, which directly translates into better quality samples when I run the reverse diffusion.

3.2 How image quality changed over training

Early on, generated samples (in the "Generated Samples" grids every few hundred steps) looked like amorphous gray blobs with no clear digit structure. After a few epochs, I started to see faint but recognizable shapes: some images clearly resembled 2s, 3s, and 7s, although they were noisy and sometimes broken. By later epochs, digits became much sharper and more consistent, strokes were continuous, backgrounds mostly uniform, and many samples would be easily recognized as

proper MNIST digits. A few failure cases remained (ambiguous digits or overly thick/thin strokes), but overall quality improved noticeably as the loss decreased.

3.3 Why time embeddings are needed

Time embeddings tell the model “how much noise remains” in the current input. At early timesteps (high noise), the model must perform large structural changes; at late timesteps, it should only perform small refinements. Without an explicit timestep input, the U-Net would receive only the noisy image and have to infer implicitly how far along the process it is, which is ambiguous. In my implementation, the scalar timestep t passes through SinusoidalPositionEmbedBlock and an MLP, then is projected with a 1×1 conv and added to the bottleneck features. This gives the model a continuous, learnable representation of the diffusion time and lets it adjust its denoising behavior appropriately at each step.

4. CLIP Evaluation

4.1 What the CLIP scores say; highest vs lowest

The CLIP scores quantify how strongly a pre-trained vision–language model associates my generated images with textual prompts like “a handwritten number 7” vs. “a blurry or unclear number”. Images with high “good” and “clear” scores (and low “blurry” scores) were typically clean, centered digits with distinct strokes. From inspecting the CLIP evaluation plots, digits such as 0, 1, and 7 often received the highest quality scores: they had strong “good/clear” probabilities and “blurry” close to zero. Some of the lowest-scoring images were noisy or malformed: for example, digits where the loop was not fully closed (so an 8 looked like a 3), where the digit was off-center or too faint, or where extra noise created spurious strokes. Those

images tended to have relatively high “blurry” scores and low “good”/“clear” scores, and their titles were rendered in red in the CLIP plots.

4.2 Hypothesis: why are some images easier/harder to generate convincingly?

My hypothesis is that digits with simple geometry and low intra-class variation (like 0, 1, and 7) are easier for the diffusion model to learn and for CLIP to recognize. They have relatively few strokes and are robust to small distortions. In contrast, digits such as 3, 5, 8, and 9 involve multiple curves and intersections, and the MNIST samples show a lot of handwriting variation. Small denoising errors, like a gap in a loop or an extra bump, can make them look like a different digit or like a scribble, which both the diffusion model and CLIP penalize. This explains why some classes produce more borderline or low-quality samples under CLIP’s evaluation.

4.3 How to use CLIP scores to improve generation (one concrete technique)

A practical way to leverage CLIP is CLIP-guided selection, which I implemented as a bonus. For each requested digit, I generate multiple candidate images (10 in my code), compute CLIP similarity scores for each sample, and define a combined quality score such as

$$\text{quality} = p(\text{good}) + p(\text{clear}) - p(\text{blurry}).$$

I then sort the samples by this quality score and keep only the top-k (e.g., top 3) as final outputs. This is a form of learned “critic” that filters out low-quality generations without retraining the diffusion model. A more advanced extension would be to incorporate a CLIP-based term into the training loss or fine-tuning stage, so that the model is directly optimized to produce images that CLIP considers clear and class-consistent.

5. Practical Applications

5.1 Real-world usefulness of this type of model

Diffusion models like the one I implemented are general-purpose generative models for images. They can be applied to many tasks: data augmentation for training classifiers (e.g., generating diverse handwritten digits or characters), denoising or restoration of corrupted images, super-resolution, inpainting (filling missing regions), and creative content generation (art, icons, logos). In more advanced settings, similar architectures are used for medical image synthesis/augmentation, generating training data for rare classes, or assisting in design workflows where many variations of an object are needed.

5.2 Limitations of my current model

1. Dataset and resolution: It is trained on MNIST-style 28×28 grayscale digits only. It cannot handle complex, high-resolution natural images.
2. Sampling speed: Generation requires running the U-Net for many timesteps (up to 1000 in my schedule), which is relatively slow compared to models with fewer steps or distilled samplers.
3. Capacity and expressiveness: The U-Net has only three scales and moderate channel sizes; some outputs are still a bit fuzzy or ambiguous.

5.3 Three specific improvements if I continued the project

1. Increase model capacity and experiment with better noise schedules.
   I would try a deeper U-Net (more down/up blocks, e.g., channel sizes (64, 128, 256, 512)) and explore cosine or sigmoid noise schedules instead of linear betas. This should

improve sharpness and stability of samples, especially for more complex digits, at the cost of extra compute.

2. Implement full classifier-free guidance.

   I would train the model sometimes *without* class conditioning (by zeroing the class embedding) and, at sampling time, combine conditional and unconditional predictions to amplify the effect of the class label. This usually makes the conditioning much stronger and yields more confidently class-correct digits.

3. Use CLIP-guided selection and/or fine-tuning.

   Building on the bonus work, I'd formalize CLIP-guided selection as part of the pipeline: always generate multiple candidates and automatically keep the top few according to CLIP. For long-term improvement, I had experiment with fine-tuning the diffusion model using a CLIP-based perceptual loss to push generations toward higher semantic quality as judged by CLIP.

Bonus Challenge: CLIP-Guided Selection (what I did and what I saw)

For the bonus challenge, I implemented CLIP-guided selection of top-3 samples per digit. For each digit 0–9 I:

1. Generated 10 candidate samples using the trained diffusion model.

2. Evaluated each sample with evaluate_with_clip, which returns three probabilities: "good handwritten digit", "clear, well-written digit", and "blurry/unclear number".

3. Defined a single quality score:

$$\text{quality} = p(\text{good}) + p(\text{clear}) - p(\text{blurry}).$$

4. Used torch.topk to select the top-3 images per digit.
5. Visualized all 10 images with the top-3 highlighted in green borders, and printed the corresponding quality scores.

Observations:

- For most digits, the top-3 CLIP-selected images were ones a human would also consider the best: they were centered, had strong contrast, and had complete strokes.
- Low-quality samples that CLIP ranked poorly tended to be off-center, overly blurred, or ambiguous between two digits (e.g., a 9 that looks like a wonky 4).
- Some digits with inherently simple shapes (0, 1, 7) produced a higher proportion of top-quality samples; more complex digits (3, 5, 8, 9) had more variation and more low-scoring outliers.

This experiment shows that CLIP can serve as an effective automatic "critic" to either filter outputs or potentially guide training towards higher-quality generations.

Source:

1 -Class review By Professor Patricia

2- Ho, Jonathan, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. Advances in Neural Information Processing Systems, 2020.